BANTAY-BAHA CODE GUIDE

A walkthrough of every file and every code block in the repository: what it does, how, and why it was written that way. Line links point at the code as of 2026-10-24; if a function has moved, search for its name.

How to read this. Section 1 is the big picture (read it first). Sections 2-9 follow the data from download to dashboard, one file at a time, in the order the pipeline runs. Each entry is one function or one top-level block.

CONTENTS

1. The big picture
2. Entry points (.bat files)
3. Getting the data: `scripts/download_data.py` , `download_gee.py` , `sync_data.py`
4. The `bantay` library
5. 4.1 config · 4.2 units · 4.3 dengue · 4.4 storms · 4.5 rain · 4.6 rivers · 4.7 grids · 4.8 flood · 4.9 panel · 4.10 leadlag · 4.11 forecast · 4.12 access · 4.13 scenarios
6. The build: `scripts/build.py`
7. The dashboard: `app/streamlit_app.py`
8. Results documents: `make_results.py` , `results_pages.py` , `results_slides.py`
9. Optional benchmark: `scripts/benchmark_tsfm.py`
10. The EDA notebook
11. Every file the pipeline writes
12. Glossary

---

1. THE BIG PICTURE

Bantay-Baha answers two questions for a typhoon track: **where will dengue surge** (R) and **who loses their route to a hospital** (A). Priority is **P = R x A**.

```
download_data.py / download_gee.py / sync_data.py  ->  data/raw/           (21+ sources, MANIFEST.csv)
                                                          |
scripts/build.py                                          v
  crosswalk  units.py          126 dengue units -> 85 harmonised provinces  (data/crosswalks/, units.gpkg)
  panel      panel.py          one row per province per week: cases, rain, rivers, storms, flood   (panel.parquet)
                uses dengue.py, storms.py, rain.py, rivers.py, grids.py, flood.py
  leadlag    leadlag.py        PCMCI+ lead-lag graph, sentinels, communities
  forecast   forecast.py       quantile XGBoost, rolling-origin test, WIS scores
  scenarios  scenarios.py      for 4 typhoons x 15 tracks: R (forecast) x A (access.py) -> P, tiers
  app        (simplified map)                                                  -> data/processed/app/
                                                          |
app/streamlit_app.py    reads data/processed/app/ only    v   dashboard
scripts/make_results.py reads data/processed/             ->  docs/results/ (report, slides, demo script)
```

Three rules shape the whole codebase:

- **Missing data never stops a build.** Every stage checks its inputs with `config.have()` / `config.missing()` ; missing inputs leave columns empty (NaN) and are listed in `data/processed/status.json` . Re-running after the files arrive picks them up with no code change. Nothing is ever filled with fake numbers.

- **No look-ahead.** Forecasts are tested by rolling origin (train on the past, predict the future, move forward); the graph and thresholds are re-learned inside each training window.
- **Heavy work happens in the build, not the dashboard.** The dashboard only reads small pre-computed files, so it answers in seconds.

2. ENTRY POINTS (.BAT FILES)

FILE	WHAT IT DOES
`setup.bat`	Team setup: creates the venv at `%USERPROFILE%\.venvs\bantay-baha` (outside OneDrive so it isn't synced), installs `requirements.txt`, registers the Jupyter kernel, signs in to Google once (`earthengine authenticate`), then runs `scripts/sync_data.py pull` to copy `data/raw/` from the team bucket. Each step is skipped if already done.
`download_data.bat`	The slow path: same venv setup, then downloads every source from its original website (`download_data.py`) and, if a GEE project id is given, the Earth Engine sources (`download_gee.py`). Used to (re)build the bucket.
`build.bat`	Runs `scripts/build.py` with the venv's Python, passing any stage names through (`build.bat panel forecast`).
`run_app.bat`	Starts the dashboard: `python -m streamlit run app/streamlit_app.py`.

3. GETTING THE DATA

scripts/download_data.py (OPEN SOURCES)

Downloads every open dataset into `data/raw/<source>/` and records each file in `data/raw/MANIFEST.csv` (source, path, bytes, sha256, url). Safe to re-run: finished files are skipped.

BLOCK	LINES	EXPLANATION
Constants `ROOT`, `RAW`, `UA`, `HDX`	23-26	Repo root and raw folder found from the script's own location (works from any working directory). `UA` is a polite User-Agent; some servers reject Python's default.
`STATIC`	29-89	The catalogue of single-file sources: `source -> [(url, filename)]`. Adding a source = adding a line here. HydroRIVERS uses the Australasia (`au`) file because the Philippines is not in the Asia file.
`EMS`, `NOAH_FOLDERS`, `NOAH_DOCS`	92-94	Sources that need scraping: Copernicus EMS activations (with a filename filter) and the UP NOAH Google Drive folder ids.
`CHIRPS_BBOX`, `CHIRPS_YEARS`	98-99	The Philippine bounding box and years for the server-side CHIRPS subset.
`PARTIAL`	101	Collects single files that failed inside a source that otherwise finished, so one bad file doesn't abort a whole source.
`fetch()`	104-125	The download primitive. Skips existing non-empty files; streams to `name.part` and renames only when complete (an interrupted run never leaves a half file that looks finished); retries 3 times with back-off; with `binary=True` refuses an HTML page (a quota or login page) masquerading as a file.
`drive_listing()`	128-130	Lists a public Google Drive folder by parsing its embedded-view HTML (no API key needed).

BLOCK	LINES	EXPLANATION
`fetch_drive()`	133-135	Downloads a Drive file via `drive.usercontent.google.com` with `confirm=t`, which skips the virus-scan page Drive shows for files over 100 MB.
`pull_static()`	138-139	Fetches every `(url, filename)` of one `STATIC` source.
`pull_ems()`	142-149	Scrapes each EMS activation page for `*_vector.zip` links and downloads the ones matching the filter (EMSR312 delineation packages; all of EMSR478).
`pull_noah()`	152-165	Downloads the NOAH readme/licence, then every province zip in the 5-yr and 25-yr folders; a failed province is recorded in `PARTIAL` and the rest continue.
`pull_chirps()`	168-176	Builds one IRI Data Library URL per year that subsets CHIRPS daily rainfall to the Philippine box on the server, then downloads it as NetCDF.
`PULLERS`	179	Maps every source name to its puller, so `main()` can run all of them or a chosen few.
`sha256()`	182-187	Checksum of a file, read in 1 MB chunks (works for multi-GB files).
`write_manifest()`	190-205	Merges new rows into `MANIFEST.csv`; recomputes the checksum only when a file's size changed (avoids re-hashing 7 GB on every run).
`main()`	208-226	Runs the selected pullers; writes the manifest after each source (an interrupted run keeps what finished); prints failures and exits non-zero if anything failed.
`if __name__ == "__main__"`	229-230	Command-line entry: source names are optional arguments.

scripts/download_gee.py (GOOGLE EARTH ENGINE SOURCES)

Needs a registered Earth Engine project (`--project <id>`). Pulls IMERG rainfall, the MAP friction surface, Global Flood Database events, a Sentinel-1 scene inventory and the Sentinel-1 flood masks.

BLOCK	LINES	EXPLANATION
Imports, `ROOT`, `RAW`, `BBOX`	17-32	Same layout as `download_data.py`; imports its `write_manifest` so GEE files land in the same manifest.
`TYPHOONS`, `IBTRACS_NAMES`	35-41	The four backtest storms: landfall dates (for before/after windows) and their IBTrACS names (to find the track).
`WRITTEN`	43	Every file written or skipped, collected for the manifest.
`download_image()`	46-61	Asks Earth Engine for a GeoTIFF of an image over a region at a scale, downloads it (unzipping when EE returns a zip), and records it. Skips files already on disk.
`pull_imerg()`	64-78	For each month 2016-2020, turns half-hourly IMERG (mm/h) into daily totals (mean x 24) and stacks the days as bands of one GeoTIFF per month.
`pull_map_friction()`	81-85	Downloads the Malaria Atlas Project 2019 friction surfaces (motorised and walking), about 1 km.
`pull_gfd()`	88-98	Writes a table of Global Flood Database events that intersect the Philippines (dates, cause, deaths). Context only: it ends in 2018.
`pull_sentinel1()`	101-120	Lists every Sentinel-1 scene from 24 days before to 14 days after each landfall (no imagery), used to plan the flood mapping.
`track_tiles()`	123-138	Picks the 1-degree tiles of the Philippine box within ~350 km of a storm's track (reads the local IBTrACS file), so we only process tiles the storm could have flooded.

BLOCK	LINES	EXPLANATION
`s1_flood_image()`	141-176	**The flood-mapping recipe**, computed on Google's servers. For each relative orbit (same satellite geometry) that imaged the tile both before and after landfall: before = median VH backscatter of days -30..-3, after = darkest smoothed image of days 0..+10; a pixel is new flood when VH drops by at least 3 dB and ends below -18 dB (water is radar-dark). Floods from all orbits are combined; permanent water (JRC), slopes over 5 degrees and specks under 8 pixels are removed. Output: 1 = new flood, 0 = dry, 255 = no data or sea. *Why not plain UN-SPIDER?* Its after/before ratio on dB values found almost none of the flood Copernicus mapped at Tuguegarao; sampling showed real 4-10 dB drops that the ratio missed.
`STORM_FILTER`, `pull_s1_flood()`	179-198	Loops over storms (or only those given with `--storms`) and their tiles; builds the flood image per tile and downloads it at 100 m. Tiles no orbit covered are reported as `none` and skipped.
`PULLERS`, main block	201-218	Command-line entry: initialises Earth Engine with the project, runs the chosen pullers and writes the manifest per source.

scripts/sync_data.py (TEAM BUCKET)

Mirrors `data/raw/` to and from the private Cloud Storage bucket, so teammates don't redo slow downloads.

BLOCK	LINES	EXPLANATION
`PROJECT`, `BUCKET`, `LOCATION`, `PREFIX`	25-28	Where the team's copy lives (Singapore region).
`client()`	31-39	Makes a Storage client from Application Default Credentials, falling back to the Earth Engine login.
`read_manifest()`, `write_manifest()`	42-50	Read/write `MANIFEST.csv` as a dict keyed by path.
`push()`	53-76	Creates the bucket if needed (private, uniform access), uploads every manifest file the bucket lacks or has a different checksum for, then uploads the manifest itself.
`pull()`	79-102	Downloads every file in the bucket's manifest that is missing locally, to `.part`, checks its sha256 and only then renames it; merges the rows into the local manifest.
main block	105-112	`python scripts/sync_data.py pull|push [sources...]`.

4. THE bantay LIBRARY

`bantay/__init__.py` only holds the package docstring. Every module below is imported by `scripts/build.py`, and some by the dashboard and results scripts.

4.1 bantay/config.py

Shared paths, constants and the missing-input registry.

BLOCK	LINES	EXPLANATION
Paths `ROOT` ... `APP_DATA`	12-17	Every folder the pipeline uses, built from the package location: raw inputs, cached intermediates (`interim`), outputs (`processed`), reviewed crosswalks, and the small files the dashboard reads (`processed/app`).
`BBOX`, `CRS_M`	19-20	The Philippine bounding box (lon/lat) and the metric projection used for distances and buffers (UTM 51N).

BLOCK	LINES	EXPLANATION
`WEEK_FREQ`, `FIRST_WEEK`, `LAST_WEEK`, `MODEL_END`	23-25	DOH weeks end on Sunday; the panel spans 2016-01-10 to 2021-01-10. `MODEL_END` (end of 2019) is the last week any model trains on or is scored on, because 2020 reporting collapsed under COVID-19.
`STORMS`	28-33	The four backtest typhoons: local name, international name, season, landfall date and the folder name used by the GEE downloads.
`QUANTILES`	36	The 23 quantile levels of the CDC FluSight forecasting hub (0.01, 0.025, 0.05, 0.10 ... 0.95, 0.975, 0.99).
`week_ending()`	39-45	Maps any date (or column of dates) to the Sunday that ends its DOH week. Used whenever daily rain or storm times are joined to weekly cases.
`_STATUS_FILE`, `_load_status()`	50-56	`data/processed/status.json` holds one line per input: available or not, a detail, and when it was checked.
`record()`	59-64	Writes or updates one input's status.
`status()`	67-68	Returns the whole status dict (the build prints it at the end; the dashboard shows it on *Data status*).
`missing()`	71-74	Prints a warning and records the input as missing; returns `None` so callers can write `return missing(...)`.
`have()`	77-86	Checks that paths exist (and, for a folder, that it holds a matching file), records the result, returns True/False.
`ensure_dirs()`	89-91	Creates the output folders.

4.2 bantay/units.py

Builds the 85 **harmonised 2016-2020 provinces** and the crosswalks into them. Needed because the dengue file has 126 reporting units (provinces mixed with 45 cities) and today's boundary file (COD-AB) uses post-2020 boundaries.

BLOCK	LINES	EXPLANATION
Paths	20-23	The COD-AB geodatabase, the two crosswalk CSVs, and the unit polygons output.
`MERGE`	26	Undoes post-2020 changes: Maguindanao del Norte/Sur back to Maguindanao (split 2022), the Special Geographic Area back to Cotabato (created 2024 from Cotabato barangays), City of Isabela into Basilan.
`SYNTHETIC_ID`	29	Maguindanao has no single current code, so it gets a clearly synthetic id.
`DENGUE_FIX`, `DENGUE_CITY_PROVINCE`	32-34	Spelling fixes for the dengue file (NUEVA VISCAYA, SISQUIJOR, BASILIG = Bislig, NORTH COTABATO) and the one ambiguous city (NAGA CITY is in Camarines Sur, from its Region V label).
`norm()`	37-41	Normalises a place name for matching: strips accents (Las Piñas -> LASPINAS), parentheses, "City of"/"City" and anything not a letter.
`read_adm()`	44-47	Reads one COD-AB level (1 = region, 2 = province, 3 = municipality) straight from the zip.
`_unit_name()`	50-54	Gives an ADM2 name its harmonised unit name and the rule used.
`build_adm2_crosswalk()`	57-64	ADM2 -> unit table. A unit's id is the official code of the member that shares its name, or the synthetic id.
`build_dengue_crosswalk()`	67-91	Matches each dengue unit to COD-AB: city units are looked up as cities first (so QUEZON CITY is not Quezon province), then province names, then municipality names; each match is folded into its unit. Ambiguous matches get a note.

BLOCK	LINES	EXPLANATION
`write_csv_with_header()`	94-99	Writes a CSV preceded by `#` comment lines that document its source and rules (the repo rule for hand-reviewed tables).
`read_crosswalk()`	102-103	Reads a crosswalk, skipping the comment lines, as text (codes keep leading zeros).
`build_crosswalks()`	106-134	Builds both crosswalks once and reuses them afterwards (unless `--force`), because they were reviewed by hand. Records how many dengue units matched (126/126).
`check_psgc()`	137-151	If the PSA PSGC Excel is downloaded, checks every COD-AB province code against it. Optional; skipped when absent.
`build_units_geometry()`	154-169	Dissolves ADM2 polygons into the 85 unit polygons, repairs invalid geometry, simplifies to ~50 m (small enough to process fast, finer than the 100 m rasters) and saves `units.gpkg`.
`load_units()`	172-175	Loads the unit polygons, building them if needed.
`adm3_with_units()`	178-184	Municipality polygons with their unit, for the municipal access results.

4.3 bantay/dengue.py

The forecast target and the population used to turn cases into incidence.

BLOCK	LINES	EXPLANATION
`DENGUE`, `CENSUS`	12-13	The two input files.
`weeks()`	16-17	The 262 Sundays from 2016-01-10 to 2021-01-10.
`load_dengue()`	20-27	Reads the DOH file, skipping row 2 (HXL tags), stripping stray spaces from names, checking that every date is a Sunday, and dropping `deaths` (2016 deaths are implausible).
`unit_week_cases()`	30-43	Builds the full unit x week grid: sums the member reporting units of each harmonised unit and keeps the total **only if every member reported** that week, so a missing city never looks like zero cases.
`unit_population()`	46-75	2020 census population per unit. The census's municipal codes don't join to COD-AB for 24 M people (old code format), but its *Province* column already places cities in their province, so units are matched on that column by name, with fixes for NCR districts, SGA and City of Isabela.

4.4 bantay/storms.py

Typhoon tracks from IBTrACS: wind radii, swaths, shifted tracks and weekly exposure.

BLOCK	LINES	EXPLANATION
`NM_KM`, `IBTRACS`, `R34`, `COLS`	18-21	Nautical miles to km, the file, the four 34-knot radius columns, and the columns read.
`load_ibtracs()`	25-33	Reads Western Pacific tracks for 2016-2020 near the Philippines (main tracks only) and picks one wind speed per fix (US, else WMO, else Japan).
`gale_radius()`	36-47	Radius of gale-force winds per fix: the largest US 34-kt quadrant, else Japan's 30-kt radius, else a fixed 150 km; fixes below gale strength get no radius. Keeps which source was used. *Why:* the US radius is missing on up to 58% of fixes.
`storm_track()`	50-60	Returns the fixes of one backtest storm (by name and season) or any storm id, in time order.

BLOCK	LINES	EXPLANATION
`shift_track()`	63-69	A "what if" track: the same storm moved by `dlat` / `dlon` degrees (and optionally in time).
`interpolate()`	72-77	Fills in hourly positions between the 3-hourly fixes, so swaths have no gaps.
`swath()`	80-92	The area that saw gale-force winds: the union of circles of the gale radius around every hourly position (in metres).
`track_line()`	95-96	The track as a line (for maps).
`unit_week_exposure()`	99-133	For every unit and week 2016-2020: the closest any storm came, whether the unit was inside a gale radius, the strongest wind within 300 km, and how many storms passed within 300 km. These become forecast features.
`scenario_exposure()`	136-150	The same exposure for one (possibly shifted) track; used to replace the storm features when a track is shifted.

4.5 bantay/rain.py

Daily rainfall grids to weekly rainfall per province.

BLOCK	LINES	EXPLANATION
`CHIRPS_DIR` , `IMERG_DIR`	18-19	Input folders.
`zonal_weights()`	22-40	A sparse matrix saying which grid cells belong to which province (cell centre inside the polygon, land cells only), each row summing to 1. Tiny islands with no cell centre get their nearest land cell. Built once, reused for every day.
`apply_weights()`	43-50	Multiplies a stack of daily grids by the weights: province mean rainfall per day, ignoring missing cells.
`_grid_transform()`	53-58	Builds the pixel-to-coordinate transform from the grid's centre coordinates, and notes whether latitude runs upwards (then the grid is flipped).
`nearest_cells()`	61-68	For a list of points (river reaches), the nearest land cell, via a KD-tree.
`chirps_daily_units()`	71-100	Reads each CHIRPS year file (with the pure-Python `scipy` NetCDF reader, because the netCDF4 library is blocked on some Windows machines), converts its julian-day time axis to dates, and returns daily province rainfall; optionally also the rainfall at each river reach.
`imerg_daily_units()`	103-122	Same for the IMERG monthly GeoTIFF stacks from Earth Engine (one band per day).
`weekly()`	125-139	Daily to DOH weeks: Monday-Sunday total (`_7d`), Friday-Sunday total (`_3d`) and wettest day (`_max1d`).

4.6 bantay/rivers.py

Rain routed downstream, so a province at a river mouth "feels" rain that fell upstream.

BLOCK	LINES	EXPLANATION
`HR_ZIP` , `HR_LAYER`	15-16	The HydroRIVERS Australasia shapefile.
`load_reaches()`	19-35	Reads river reaches in the Philippine box and assigns each to the unit containing its midpoint (reaches just off the coastline go to the nearest unit within 2 km). Sorted by upstream area, which puts every reach before the reaches downstream of it.
`accumulate()`	38-47	Walks reaches from upstream to downstream, adding each reach's rain volume (rain x local catchment area) to the next reach down. Dividing by upstream area gives the average rain over everything upstream of each reach.

BLOCK	LINES	EXPLANATION
`unit_week_river()`	50-69	Weekly version per unit: discharge-weighted average of upstream rain over the unit's reaches (`river_up_7d`) and how much more that is than the unit's own rain (`river_excess_7d`).

4.7 bantay/grids.py

Population grids aligned with WorldPop (100 m), so population, flood and hazard line up cell for cell.

BLOCK	LINES	EXPLANATION
Constants	16-19	WorldPop file, the unit-id raster, the 1 km population cells, and the processing block size.
`unit_index()`	22-25	Gives each unit a small integer code (1-85) for rasters; 0 means outside every unit.
`build_unit_raster()`	28-46	Burns the unit polygons into a raster on the exact WorldPop grid, block by block (the full grid is 230 M cells).
`iter_blocks()`	49-60	Walks several aligned rasters together, one block at a time, to keep memory low.
`worldpop_clean()`	63-66	Turns nodata and negative values into 0 people.
`unit_population_worldpop()`	69-79	Population per unit from WorldPop; the fallback when the census table is missing.
`build_pop_cells()`	82-128	Sums WorldPop into ~1 km cells (369,121 cells, 109 M people), gives each its unit (centre cell, or any unit in the block for coastal cells) and its municipality (containing polygon, else nearest). These are the "people" the access model routes to hospitals.

4.8 bantay/flood.py

Flood exposure: UP NOAH hazard maps (static), Copernicus EMS and Sentinel-1 (observed, per typhoon).

BLOCK	LINES	EXPLANATION
`NOAH` , `EMS` , `S1` , `RPS`	27-30	Input folders and return periods (5-yr, 25-yr).
`hazard_raster_path()`	33-34	Where the national hazard raster for a return period is cached.
`_nk()`	39-40	Name normaliser for NOAH file names.
`noah_file_units()`	43-67	Works out which units each NOAH zip covers, from its shapefile names (CamelCase province names or old 9-digit codes) including multi-province files (Mindoro, Metro Manila, pre-split Maguindanao).
`_hazard_col()`	70-71	Finds the hazard-class column (usually `Var`).
`build_hazard_raster()`	74-121	One national raster per return period on the WorldPop grid: **255 = no map, 0 = mapped and safe, 1-3 = low/medium/high hazard**. Starts all-255, then for each province file marks its units as mapped and burns in the hazard polygons (highest class wins). Rebuilt only when a NOAH file is newer. *Why 255:* "no map" must stay missing, never become "no hazard".
`unit_hazard_shares()`	124-154	Share of each unit's people in any hazard class and in the high class; NaN when less than half the unit's people are on a mapped area (11 units have no 25-yr map).
`ems_observed_flood()`	159-175	Reads the observed-flood polygons from the Copernicus EMSR312 packages (Ompong; first map and later monitoring map).
`s1_tiles()`	180-182	Lists a storm's Sentinel-1 flood tiles.
`s1_unit_flood()`	185-213	Per unit: share of the land Sentinel-1 saw that newly flooded, and people in that flood (WorldPop). Each tile is resampled onto the WorldPop grid.

BLOCK	LINES	EXPLANATION
`s1_flood_lookup()`	216-238	Loads a storm's tiles into memory once and returns a function that answers "flooded or not" for millions of points at once with array indexing (0.6 s for 2.1 M road segments; the first version took 44 s).

4.9 bantay/panel.py

Step 0: the one table every model reads (85 units x 262 weeks).

BLOCK	LINES	EXPLANATION
`PANEL` , `CARD`	15-16	Output files: the panel and its data card.
`COLUMNS`	18-43	Every panel column with its source and meaning; it drives the data card and guarantees every documented column exists (NaN when its input is missing).
`_newest()` , `cached()`	46-60	A small cache: reuse an intermediate in `data/interim/` if it is newer than all its inputs, else rebuild it. Missing inputs (`None`) are never cached, so they are retried next time.
`rain_layers()`	63-88	CHIRPS weekly rain plus river routing in a single pass over CHIRPS (the slow part), and IMERG weekly rain; both cached.
`static_layers()`	91-97	Builds the unit raster and NOAH hazard rasters (first run only) and returns hazard shares per unit.
`s1_layers()`	100-112	Sentinel-1 flood per unit, placed in each storm's landfall week and the 3 weeks after. Rolly and Ulysses landed 10 days apart, so their windows overlap; the larger flood per unit-week is kept.
`build_panel()`	115-162	Assembles the panel: the full unit x week grid, cases, population (census, else WorldPop), incidence per 100,000, storm exposure (weeks with no storm nearby get "far": 2000 km, no hit), rain, rivers, hazard, the storm-x-hazard interaction and Sentinel-1. Writes the parquet and the data card.
`write_card()`	165-178	Writes `panel_card.md` : coverage of every column and the status of every input.
`load_panel()`	181-182	Reads the panel.

4.10 bantay/leadlag.py

Layer 2: which provinces' dengue rises first, and which follow.

BLOCK	LINES	EXPLANATION
`TAU_MAX`	16	Longest lead considered: 3 weeks.
`prepare()`	19-28	For a training window: log incidence, minus the same week a year earlier (removes the shared rainy-season peak, which would otherwise link every pair), drops units with over 10% missing weeks, fills short gaps, standardises each series.
`_bh()`	31-38	Benjamini-Hochberg false-discovery correction of p-values.
`xcorr_graph()`	41-60	The simple baseline: correlation between province i lagged 1-3 weeks and province j now; keeps positive links that survive false-discovery control. Gives 4,265 links, mostly the shared 2019 epidemic.
`pcmci_graph()`	63-88	PCMCI+ (Tigramite, partial correlations): a link i -> j is kept only if it holds after conditioning on j's own past and other provinces. Keeps lagged links between different provinces with q < 0.05 (on pure noise, PCMCI+ alone returned 60-90 false links). Optional bootstrap gives each link a confidence.

BLOCK	LINES	EXPLANATION
`node_metrics()`	91-107	PageRank on the reversed graph (high score = many provinces follow you: a sentinel), number of leaders and followers, and Louvain communities (provinces that move together).
`learn()`	110-117	Runs the above for a window ending at a given date and returns links, per-province metrics and a summary (series, weeks, link count).

4.11 bantay/forecast.py

The forecaster, its baseline, the scores and the rolling-origin test.

BLOCK	LINES	EXPLANATION
`Q` , `IQ50` , `HORIZONS` , `SURGE_PCTL`	21-24	The 23 quantiles, the index of the median, horizons 1-4 weeks, and the surge definition (above the unit's 90th percentile).
`GROUPS` , `GRAPH_FEATURES`	26-37	Every feature, grouped into drivers shown on the dashboard (recent cases, season, rain, river, storm/flood, outbreak graph, unit).
`base_features()`	42-79	Features known at the forecast week t (never later): log incidence now and 1-3 weeks ago, 4- and 8-week averages, 4-week change, rain lags and 4/8-week totals, river rain, storm exposure over the last 4 weeks, hazard shares, Sentinel-1 flood, population, and the province as a category. Features whose source is missing are left out.
`target_frame()`	82-93	Adds the target for horizon h (log incidence at t+h), the same week last year (the seasonal-naive value) and the week-of-year of the target.
`cum4_frame()`	96-104	Target = total incidence over the next 4 weeks (used for the surge probability R).
`graph_features()`	107-127	For each follower province, its leaders' recent rise and year-on-year anomaly (weighted by link strength), and the number of leaders. Uses only leader values up to week t.
`feature_columns()`	130-134	The feature list, with or without the graph features (the ablation).
`fit_quantile_model()`	139-144	One XGBoost model that outputs all 23 quantiles at once (quantile loss), 150 trees, depth 4.
`predict_quantiles()`	147-148	Predicts and sorts each row's quantiles (so they never cross).
`usable()`	151-153	Drops features that are empty in the training rows.
`seasonal_naive()`	156-160	The bar to beat: "same week last year", with a range taken from its past errors.
`prob_exceed()`	163-175	Turns 23 quantiles into the probability of exceeding a threshold, by interpolating the quantile curve.
`surge_thresholds()`	178-183	Each unit's 90th percentile of 4-week incidence, computed from training weeks only.
`to_cases()`	188-189	Back from log incidence per 100,000 to case counts.
`wis()`	192-202	The weighted interval score (Bracher et al., 2021): rewards narrow ranges that contain the truth and penalises misses in proportion to how far out they are.
`coverage()`	205-208	Whether the outcome fell inside the 50% or 90% range.
`refit_dates()`	213-214	Refit every 13 weeks from January 2018 to December 2019 (8 refits).
`rolling_origin()`	217-254	**The honest test.** At each refit date: re-learn the graph and thresholds on past data, train on rows whose target is before the refit date, forecast every week until the next refit. Fits the model with and without graph features; when a window's graph has no links, the two are identical, so the second fit is skipped.

BLOCK	LINES	EXPLANATION
`score()`	257-285	WIS, error and coverage of every forecast for all three models, plus an online conformal 90% band (built only from errors of forecasts already past) as a check.
`summarise()`	288-304	Averages by horizon and overall: WIS, relative WIS vs seasonal naive, graph vs no-graph, coverage.
`surge_calibration()`	307-319	Brier score of R, the base rate, and a reliability table (when R says x, how often did a surge happen).
`fit_at()`	324-338	Trains all models on data up to a storm's landfall week (never past 2019), for the scenarios.
`predict_at()`	341-368	Forecasts every province from that week; optional overrides replace the storm features (shifted tracks); computes R and the per-driver SHAP contributions shown on the dashboard.

4.12 bantay/access.py

Layer 3: travel time to the nearest hospital over roads and ferries, before and during a storm.

BLOCK	LINES	EXPLANATION
`PBF` , `ROADS` , `SPEED`	31-38	The OpenStreetMap file, the cached road graph, and assumed travel speeds per road class (links at 80% of their road). Speeds are a tunable assumption, tested in the sensitivity analysis.
`WALK_KMH` , `MAX_SNAP_KM` , `LAT0`	38-39	Walking 5 km/h from a home to the road, at most 5 km; a reference latitude for fast km conversions.
`_xy()` , `_hav_km()`	42-50	Coordinates to km (for nearest-node search) and great-circle distance.
`build_road_graph()`	55-92	Reads OSM roads through GDAL, finds junctions as coordinates shared by two or more roads (OSM roads share nodes there), cuts roads into segments between junctions, and stores each segment's length, speed and midpoint. 1.72 M junctions, 2.13 M segments, cached.
`_nhfr()`	97-118	Reads the DOH facility registry if downloaded by hand: finds latitude/longitude and type columns and keeps hospitals.
`load_hospitals()`	121-150	NHFR if available, else the fallback: OSM hospitals plus NOAH facilities whose names say hospital or medical center, with duplicates within 300 m removed (2,199). Labelled as a fallback everywhere.
`load_ferries()`	155-164	The 108 RoRo routes; crossing time plus half the time between departures (capped at 12 h), and a straight line between ports for storm checks.
`class Network`	169-220	The routable network.
`__init__`	172-195	Snaps hospitals, population cells and ferry ports to the nearest road junction (ferry ports within 25 km, because their coordinates are coarse).
`road_hours()`	197-202	Travel hours per segment, with optional speed multipliers (0 = road closed).
`solve()`	204-220	Builds a sparse graph of roads and ferries (keeping the fastest of parallel edges) and runs one multi-source Dijkstra from all hospitals at once: hours from every junction to its nearest hospital. Adds the walk from each population cell. One national solve takes about 4 s.
`cut_off()`	223-225	A cell is cut off if it had a route before the storm and now has none, or its trip at least doubled and grew by more than an hour.

BLOCK	LINES	EXPLANATION
`ferry_keep_mask()`	230-239	Which ferry routes still sail: cancelled if the route crosses the gale swath (or, in the sensitivity variant, if a port is inside it).
`edges_in()`	242-249	Which road segments lie inside the swath (vectorised point-in-polygon).
`hazard_at()`	252-271	NOAH hazard class at each segment midpoint, read in bands to save memory.
`road_multiplier()`	274-284	Speed multiplier per segment: observed flood closes roads; otherwise (shifted tracks) high hazard inside the swath closes and medium hazard slows. Returns which rule was used.
`FRICTION`, `friction_times()`	289-317	The second access method, on the MAP friction raster (time to cross each 1 km cell): cost-distance from hospitals. Written and tested on load, not yet wired into the scenarios.
`summarise()`	322-334	Per unit (or municipality): population, people cut off, people with no route even normally, average normal trip time, and A = share cut off.

4.13 bantay/scenarios.py

Puts it together: for each typhoon and track, R x A = P, action tiers, then the checks.

BLOCK	LINES	EXPLANATION
`SHIFTS`	24	The 15 tracks per storm: north/south 0, ±0.5, ±1 degree x east/west 0, ±1.
`TIERS`, `TIER_LABELS`	25-28	Provisional action rules: deploy if R >= 0.5 and A >= 10%; pre-position if R >= 0.3, or A >= 10% and R >= 0.15; else monitor.
`tier()`	31-36	Applies the rules to every province.
`overrides_for()`	39-45	Storm features for a shifted track (gale hit, wind, distance, storm x hazard), to replace the real storm's in the forecast. Observed rain is kept.
`class AccessModel`	48-105	Builds the network once and reuses it for all 60 scenarios.
`__init__`	51-63	Road graph, hospitals, ferries, the normal-day travel times, hazard per segment and EMS polygons. If an input is missing, A stays NaN.
`observed_flood()` / `_observed_flood()`	65-84	Which road segments were really flooded for a historical storm: Sentinel-1 if available, else EMS for Ompong. Cached per storm.
`run()`	86-105	One storm scenario: cancel ferries in the swath, cut flooded roads (observed for the real track, hazard proxy for shifted tracks), re-solve, mark who is cut off. Options are the sensitivity knobs.
`run_all()`	108-166	For each storm: forecast origin = landfall week, models trained to that week; for each of 15 tracks: R and expected cases (median and 90% range), A per province and municipality, P, tier, sentinel status; saves everything the dashboard needs.
`validate()`	169-190	Backtest: did provinces with high R / P see more cases in the next 4 weeks? Spearman correlations, surge rate of the top 10, Brier score. 2020 storms are flagged.
`sensitivity()`	193-218	Re-runs access under 9 changed assumptions (speeds, ferry rule, slow-down, cut-off rule, storm radius) and measures how much the ranking moves (Kendall's tau, top-10 overlap).
`metro_manila_benchmark()`	221-234	Closes every NCR road in medium/high flood hazard and compares the share cut off (30%) with the published ~22% (Tariverdi et al., 2023).

5. THE BUILD: scripts/build.py

One command runs the stages in order; each can also be run alone.

BLOCK	LINES	EXPLANATION
Imports and warning filters	20-38	Adds the repo root to the import path (so `bantay` imports from any folder) and silences known harmless warnings.
`STAGES`	39	The stage order: crosswalk, panel, leadlag, forecast, scenarios, app.
`stage_crosswalk()`	42-47	Crosswalks (reused if present), the optional PSGC check, unit polygons. Stops if COD-AB is missing, because everything depends on it.
`stage_panel()`	50-52	Builds the panel.
`stage_leadlag()`	55-64	The final graph on all 2016-2019 data (PCMCI+ with 20 bootstrap samples, and the correlation baseline), saved for the dashboard.
`stage_forecast()`	67-93	Runs the rolling-origin test and saves the scores, the calibration and compact forecasts (5/25/50/75/95% in cases) for the dashboard's fan charts.
`stage_scenarios()`	96-116	Population cells, hazard shares, all scenarios, validation, sensitivity and the Metro Manila benchmark.
`stage_app()`	119-123	A lighter province map for the dashboard.
main block	126-145	Parses stage names and options (`--force` , `--boot` , `--refit-weeks`), runs the stages with timings, then lists every missing input.

6. THE DASHBOARD: app/streamlit_app.py

Streamlit re-runs the whole script top to bottom every time the user changes a control; `@st.cache_data` keeps the loaded files in memory, so each re-run is fast. **Restart it after a rebuild** to drop the cache.

BLOCK	LINES	EXPLANATION
Imports, page setup	1-23	Adds the repo root to the path and sets a wide page.
`SEQ` , `TIER_COLOR` , `TIER_SHORT` , colours	25-28	A single-hue blue scale for magnitudes, blue steps for the three tiers (always shown with a text label), orange/red for track, swath and cancelled ferries.
`FLOOD_LABEL`	29-31	Readable names for the flood source used on roads.
`DRIVERS`	32	The driver groups whose SHAP values explain each risk.
`read()`	35-43	Reads a pre-computed file if it exists (parquet, CSV or JSON), else returns None, so a missing output hides its panel instead of crashing.
`load()`	47-62	Loads every output once (cached): scenarios, municipal results, tracks, swaths, ferries, graph, evaluation, validation, sensitivity, benchmark, map and status.
Guard	65-68	If no scenarios exist yet, tells the user to run the build and stops.
`names`	70-72	Unit id to province name, from the map file.
Sidebar	75-81	Typhoon picker, track shift sliders (north/south, east/west) and map colour (P, R or A).
Scenario selection	83-87	Picks the 85 rows of the chosen storm and shift.

BLOCK	LINES	EXPLANATION
Header and caveats	89-107	Title, forecast origin, training cut-off, flood source, ferries cancelled, hospital source; then context notes: 2020 reporting collapse, missing access, the hazard proxy, the Sentinel-1 lower bound, shifted tracks, ferry routes as a proxy.
KPIs	112-117	Provinces at "deploy" and "pre-position", people cut off, top priority.
Tabs	119	The eight tabs.
Priority map	123-165	SVG map (no map tiles, works offline) coloured by P/R/A, with the gale swath, the track (hover: time and wind) and cancelled ferry routes.
Risk vs access	168-187	Scatter of R against A, bubble size = population, colour + label = tier, threshold lines, top six named.
Ranked provinces	190-209	The table: action, P, R, A, cases next 4 weeks with 90% range, people cut off, sentinel rank, community, the two strongest drivers; CSV download.
Municipalities	212-223	For a chosen province: each municipality's people cut off, share, normal trip time and people with no route even normally.
Outbreak graph	226-244	The PCMCI+ links with confidence, sentinel ranking, communities, and the correlation-baseline count for contrast.
Forecast skill	247-282	The evaluation table, R's reliability chart, and a fan chart (90% range, median, seasonal naive, reported cases) for any province and horizon.
Validation	285-298	Backtest table, sensitivity (Kendall's tau per variant and storm), Metro Manila benchmark.
Data status	301-305	Every input with available / missing and the detail, from `status.json`.

7. RESULTS DOCUMENTS

scripts/make_results.py

Draws every figure from the build outputs, writes the HTML for the report, slides and demo script, prints them to PDF with headless Chrome/Edge, and checks for layout overflow.

BLOCK	LINES	EXPLANATION
Paths and palette	20-42	Output folders and colours (teal, amber, red, grey; checked for colour-blind separation), and the teal map scale.
`load()`	47-72	Reads every output the documents use.
`stats()`	75-127	Computes every number quoted in the documents from the outputs (files, sources, coverage, scores, graph size, access, storm table, top priorities, Sentinel-1 per storm), so the text can't drift from the data.
`s1_ems_agreement()`	130-160	Compares Sentinel-1 with Copernicus EMS inside each EMS area: area each mapped, overlap, precision (55%) and recall (24%).
`style()`, `mono()`, `save()`	165-187	Registers the bundled fonts with matplotlib, sets a clean chart style, monospace tick labels, and saves PNGs at 220 dpi (fonts baked in).
`fig_timeline()`	192-208	National weekly dengue with the four typhoons and the 2020 collapse marked.
`fig_coverage()`	211-230	Share of panel cells with a value, per input.

BLOCK	LINES	EXPLANATION
`fig_wis()`	233-266	WIS by horizon for the three models (with relative WIS labels) and coverage against the 80-95% target.
`fig_reliability()`	269-282	R's reliability curve.
`fig_fan()`	285-304	Rolling-origin forecasts for Cagayan, 1 and 4 weeks ahead.
`_basemap()`	307-311	Grey province outlines and the Philippine extent for maps.
`fig_leadlag()`	314-335	The PCMCI+ links as arrows on the map (NCR districts share one label).
`fig_priority()`	338-358	Ompong's P map with track, swath and top-5 labels.
`fig_access_multiples()`	361-377	A per province for the four typhoons on one colour scale.
`fig_scatter()`	380-400	Ompong's risk vs access, marker shape = tier.
`fig_validation()`	403-419	Backtest correlations per storm.
`fig_s1()`	422-443	Sentinel-1 new-flood share per province in each landfall week.
`fig_s1_ems()`	446-466	Sentinel-1 flood vs EMS outlines around Tuguegarao.
`fig_network()`	469-481	Ferry routes and hospitals over the islands (the access problem slide).
`fig_rolling()`	484-502	Diagram of the 8 rolling-origin refits (train on the past, score the next block).
`make_figures()`	505-509	Draws them all.
`FONT_CSS`	514-531	The bundled fonts (Big Shoulders Display, Barlow, Space Mono) and colour variables shared by all three documents.
`REFERENCES`	532-565	The APA 7 reference list used by the report and slides.
`fmt_m()` … `source_rows()`	566-651	Small helpers that turn outputs into HTML table rows (storms, evaluation, links, sensitivity, validation, data sources).
`browser()` , `to_pdf()`	656-671	Finds Chrome or Edge and prints an HTML file to PDF headlessly.
`SCRIPT_CSS` , `_md_inline()` , `demo_script_html()`	674-727	Turns the demo-script Markdown (the editable source) into a styled landscape PDF.
`OVERFLOW_JS` , `check_overflow()`	730-772	Lays each page out in headless Chrome and lists any element that runs into the footer band or off the edge, so overflow is caught automatically.
`main()` and entry	775-802	Loads, computes, draws, writes the HTML, checks overflow, prints the PDFs. `--no-pdf` skips printing.

scripts/results_pages.py (EVALUATION REPORT, A4)

BLOCK	LINES	EXPLANATION
`REPORT_CSS`	10-52	Page layout: A4 pages with a dark side rail (page number and section), condensed headings, tables, notes, the cover.
`_page()`	54-56	Wraps one page's content with its rail and footer.
`_targets()`	59-71	The scorecard rows: each proposal objective, the result and met / partly / not yet.

BLOCK	LINES	EXPLANATION
`report_html()`	74-324	All pages in order: cover, summary, data, panel, methods, forecast, ablations, graph, Sentinel-1 (when present), access, Ompong priority, validation, scorecard, limitations, references. Text that depends on whether Sentinel-1 exists switches automatically; figures are numbered in page order at the end.

scripts/results_slides.py (20 SLIDES, 16:9)

BLOCK	LINES	EXPLANATION
`SLIDE_CSS`	11-83	Slide layout: dark slate background, amber signal tags, white chart cards, the pipeline diagram (grid with arrows that touch the boxes), timelines, stat blocks.
`N`	85	Slide count (checked at the end).
`_slide()`	88-91	Wraps a slide: signal tag numbered with the slide (Signal 01-20), the judging criterion it answers, brand and page number.
`_img()`	94-95	A figure on a white card at a set height.
`slides_html()`	98-312	The 20 slides: title with headline results; the problem (typhoons and disease, the cascade, access in an archipelago); design; data; data challenges; method; eight result slides (forecast skill, a forecast in a storm year, surge probability, outbreak links, Sentinel-1 flood, access loss, the Ompong case, validation); the demo; deployment in the loop; usability; sources. No names or internal file names (anonymous judging).

8. OPTIONAL BENCHMARK: scripts/benchmark_tsfm.py

BLOCK	LINES	EXPLANATION
`main()`	26-57	Runs the Chronos-2 foundation model zero-shot on the same rolling-origin weeks (each province's history up to the forecast week only) and scores it like XGBoost. Needs `requirements-bench.txt` (torch); if missing, records that and exits. Not run yet.
`_isfloat()`	60-65	Picks the quantile columns out of Chronos' output.
entry	68-71	`--device cuda` by default.

9. THE EDA NOTEBOOK

`notebooks/00_data_eda.ipynb` checks each raw dataset before any modelling: does it exist, what does it cover, and does it match what the proposal assumed. Findings are collected with `note(dataset, msg)` and shown in one table at the end.

CELL	SECTION	WHAT IT CHECKS
0-1	Setup	Dataset list; imports, paths, the `note()` and `have()` helpers, a summary of `MANIFEST.csv` by source.
2-4	1. DOH dengue	Rows, weeks (all Sundays), 126 units incl. 45 cities, cases and deaths per year (2016 deaths implausible, 2020 collapse), national curve and missing-unit share.
5-7	2. CCHAIN	City-week disease totals; leptospirosis series for 12 cities, 2008-2022.
8-9	3. Mendeley	National weekly diseases 2019-2023; coverage per year.
10-11	4. OpenDengue	Philippine series by space/time resolution; correlation with the DOH national total (r = 0.95).

CELL	SECTION	WHAT IT CHECKS
12-15	5. Boundaries, census	COD-AB layers and versions; the name match of the 126 dengue units (the origin of the crosswalk logic); census sheets.
16-17	6. IBTrACS	The four storms, fixes, max wind, how often the 34-kt radius is present; track map.
18-19	7. Rainfall	CHIRPS days, missing days, julian time axis; IMERG files.
20-23	8. Flood	EMS packages and EMSR312 flood area; NOAH files and which provinces have maps; GFD and Sentinel-1 inventory.
24-25	9. HydroRIVERS	Reaches touching the Philippines and outlets.
26-29	10. Access	roroph routes and coordinate coarseness; OSM roads and ferries; health facilities; friction raster.
30-31	11. Population	WorldPop total (109.6 M) and Meta HRSL layers.
32-33	Summary	All findings in one table.

10. EVERY FILE THE PIPELINE WRITES

FILE	WRITTEN BY	READ BY
`data/raw/**`, `data/raw/MANIFEST.csv`	download scripts, `sync_data.py`	everything
`data/crosswalks/adm2_to_unit.csv`, `dengue_loc_to_unit.csv`	`units.build_crosswalks` (then reviewed)	all later stages
`data/processed/units.gpkg`	`units.build_units_geometry`	panel, storms, maps
`data/interim/units_100m.tif`, `noah_*_hazard_100m.tif`	`grids`, `flood`	hazard shares, access, Sentinel-1
`data/interim/rain_*_unit_week.parquet`, `storm_unit_week.parquet`	`panel` (cached)	`panel`
`data/interim/road_graph.npz`, `pop_cells_1km.parquet`	`access`, `grids`	scenarios
`data/processed/panel.parquet`, `panel_card.md`	`panel.build_panel`	leadlag, forecast, scenarios, results
`data/processed/status.json`	`config.record` (every stage)	build summary, dashboard
`data/processed/app/leadlag_*`	build `leadlag` stage	dashboard, scenarios, results
`data/processed/app/eval_*`, `surge_*`	build `forecast` stage	dashboard, results
`data/processed/app/scenarios*.parquet`, `tracks`, `swaths`, `ferries*`, `hospitals`, `validation.csv`, `sensitivity.csv`, `benchmark_ncr.csv`	build `scenarios` stage	dashboard, results
`data/processed/app/units_simplified.geojson`	build `app` stage	dashboard, results
`docs/results/*.pdf`, `figures/`, `src/`	`make_results.py`	people

11. GLOSSARY

TERM	MEANING
R	Surge probability: chance the next 4 weeks' incidence exceeds the province's 90th percentile.
A	Access loss: share of a province's people who lose their route to a hospital in the storm.
P	Priority = R x A.
Unit	One of the 85 harmonised 2016-2020 provinces (NCR is split into its 4 districts).
Incidence	Cases per 100,000 people.
Rolling origin	Testing a forecaster as if in real time: train on the past, forecast forward, move the cut-off forward.
WIS / relative WIS	Weighted interval score of a forecast range (lower is better); relative WIS < 1 means better than the seasonal-naive baseline.
Seasonal naive	"Same week last year", the baseline forecast.
Quantile	A forecast at a probability level; 23 of them describe the full range.
Coverage	How often the truth falls inside a stated range (90% range should catch ~90%).
Brier score	Error of a probability forecast (lower is better).
PCMCI+	A causal-discovery algorithm for time series that keeps a link only if it survives conditioning on other series and their pasts.
FDR / Benjamini-Hochberg	A correction that controls the share of false links when testing thousands of pairs.
Sentinel / PageRank / Louvain	A province others follow (scored with PageRank); groups of provinces that move together (Louvain).
Gale swath	Area that saw gale-force (34-knot) winds along the track.
Shifted track	The same storm moved north/south/east/west, for "what if" scenarios.
NOAH hazard proxy	Using flood-hazard maps instead of observed flood to decide which roads close; an upper bound.
Sentinel-1	Radar satellite that sees flood through clouds; our observed-flood source (a lower bound).
EMS	Copernicus Emergency Management Service flood maps; used to check Sentinel-1.
Dijkstra	Shortest-path algorithm; run from all hospitals at once to get each place's nearest-hospital time.
Crosswalk	A table mapping one set of place codes or names onto another.
Manifest	`data/raw/MANIFEST.csv` : every raw file with size and checksum.